AutoCloud-Agent: Multi-Agent Reinforcement Learning for Cloud Resource Management

K Chaitanya (25CS60R28)
Rahul K (25CS60R16)
S.L.R. Siddesh Devarakonda (25CS60R80)

Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur
April 2026

***Abstract*—Cloud autoscaling requires balancing two competing objectives: *elasticity* (responding to workload changes quickly enough to meet SLAs) and *cost minimization* (not running more VMs than necessary). Existing approaches—reactive threshold rules (Kubernetes HPA, AWS Auto Scaling) and monolithic RL agents—suffer from delayed responses and intractable action spaces, respectively. We present AutoCloud-Agent, a multi-agent reinforcement learning system that decomposes cloud resource management into three concurrent sub-problems—horizontal scaling, VM consolidation, and job scheduling—each handled by an independent PPO agent operating at a distinct temporal frequency. A Transformer-based workload forecaster with Monte Carlo (MC) Dropout provides uncertainty-aware demand predictions, enabling proactive scaling decisions. A rule-based Safety Coordinator enforces seven hard constraints (minimum node floor, boot protection, uncertainty-driven suppression, queue emergency scale-out) to guarantee system stability regardless of policy quality. Evaluated on the Alibaba Cluster Trace 2018 (4,023 machines, 7 days), AutoCloud-Agent achieves 96.2% cost efficiency and 55.2% CPU utilization with 100% SLA compliance, matching the best classical method (MPC) while outperforming Kubernetes HPA by 3.4% on cost and 77% on CPU utilization. Ablation studies confirm that multi-agent decomposition improves cost efficiency by 4.1% and CPU utilization by 33.7% over a single-agent PPO baseline, validating the architectural choice.**

***Index Terms*—Cloud computing, autoscaling, reinforcement learning, multi-agent systems, resource management, elasticity, cost optimization, safety constraints**

I. Introduction

Cloud computing providers operate large-scale clusters of virtual machines (VMs) to serve diverse workloads with varying resource demands and latency requirements. The fundamental autoscaling challenge is deciding, at every point in time, (i) how many VMs should be active, (ii) which idle VMs should be shut down, and (iii) how incoming jobs should be assigned to available resources—all while minimizing operational cost and meeting service level agreements (SLAs).

This problem is characterized by two competing objectives:

1) **Elasticity:** The system must scale resources up quickly when demand spikes to prevent SLA violations (e.g., P95 latency exceeding 500 ms). VM boot latency (30–90 s) makes this challenging: by the time a reactive system detects overload, the SLA window may already be breached.
2) **Cost Minimization:** The system must scale down aggressively during low-demand periods to avoid paying for idle resources. Over-provisioning even a few VMs for several hours can increase cluster costs significantly.

Traditional autoscaling systems such as Kubernetes Horizontal Pod Autoscaler (HPA) [8] and AWS Auto Scaling [9] use reactive threshold-based rules (e.g., "if CPU $>$ 80%, add a node"). These approaches have two limitations: (1) they respond *after* demand increases, incurring latency during the VM boot window, and (2) they treat scaling, consolidation, and scheduling as independent problems, missing coordination opportunities.

Recent work has applied reinforcement learning (RL) to autoscaling [4], [5], [11], [12], demonstrating that learned policies can outperform hand-tuned rules. However, most RL approaches use a single monolithic agent that must control all resource management decisions simultaneously, leading to a combinatorially large action space ($3 \times 2^{20} \times 5 \approx 15$ million for our problem).

We propose **AutoCloud-Agent**, which addresses these limitations through three key contributions:

1) **Multi-agent decomposition (I-PPO):** Three independent PPO agents handle scaling, consolidation, and scheduling at different temporal frequencies, reducing the joint action space from $\sim$15 million to three manageable sub-spaces of size 3, 2^{20}, and 5.
2) **Uncertainty-aware forecasting:** A Transformer encoder with MC Dropout [3] predicts future demand and quantifies prediction confidence, enabling proactive scaling *before* demand materializes.
3) **Safety-constrained execution:** A rule-based Safety Coordinator enforces seven hard constraints as a post-processing filter, guaranteeing system stability regardless of agent policy quality—an important property absent from most RL-based autoscalers.

II. Related Work

A. Threshold-Based Autoscaling

Kubernetes HPA [8] computes the desired number of replicas as $\lceil \text{currentReplicas} \times \frac{\text{currentCPU}}{\text{targetCPU}} \rceil$, with a 10% dead-band to prevent oscillation and a configurable 5-minute stabilisation window for scale-down. AWS Target Tracking [9] uses a similar formula with asymmetric cooldown periods (300 s for scale-down, 60 s for scale-up) and CloudWatch metric aggregation. Google Cloud Autoscaler [10] adds predictive capabilities but remains fundamentally rule-based. These methods are widely deployed and operationally simple but purely reactive—they cannot anticipate demand changes, leading to SLA violations during the VM boot window.

B. Model Predictive Control for Cloud Scaling

Model Predictive Control (MPC) [18] optimises scaling decisions over a finite planning horizon using a demand forecast model. Zhang et al. proposed AWARE [4], which applies MPC with exponentially weighted moving average (EWMA) forecasts in production cloud systems, demonstrating significant improvements over reactive baselines at USENIX ATC 2023. Lorido-Botran et al. [17] survey auto-scaling techniques and find that MPC-based approaches consistently improve on threshold methods by 15–25% on cost metrics. However, MPC requires an accurate demand model and hand-tuned objective weights, limiting adaptability to non-stationary workloads.

C. Reinforcement Learning for Resource Management

Mao et al. [5] introduced DeepRM, applying deep RL to cluster resource management by learning to pack jobs onto machines. Qiu et al. [11] proposed FIRM, an RL-based autoscaler for microservices that learns scaling policies from application-level SLOs. Tirmazi et al. [12] analysed Google's Borg traces, providing insights into workload patterns that motivate learned approaches. However, most RL approaches use a single agent with a flat action space, limiting scalability. Multi-agent RL (MARL) has been applied successfully to other cooperative control problems [13], and Independent PPO (I-PPO) [14] has shown competitive performance in cooperative settings. We adapt I-PPO to cloud autoscaling with temporal hierarchy—a novel contribution.

D. Uncertainty Estimation in Neural Networks

Gal and Ghahramani [3] showed that MC Dropout approximates Bayesian inference, providing calibrated uncertainty estimates via multiple stochastic forward passes. Lakshminarayanan et al. [15] proposed deep ensembles as an alternative but at $N\times$ the compute cost. We choose MC Dropout (30 passes through one model) over ensembles (5–10 separate models), achieving comparable uncertainty quality at $\sim 5\times$ lower compute—a practical consideration for real-time autoscaling where inference latency matters.

Schulman et al. [1] introduced Proximal Policy Optimization (PPO), which we use as the base RL algorithm. PPO clips the policy update ratio to prevent destructive large updates, providing stable training even with function approximation.

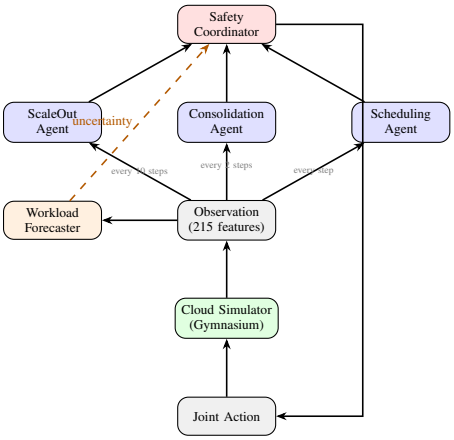

Fig. 1: AutoCloud-Agent system architecture. Three I-PPO agents operate at different temporal frequencies. The Safety Coordinator filters all actions before execution. Dashed arrow indicates the forecaster's uncertainty signal σ that drives proactive scaling decisions.

III. System Architecture

AutoCloud-Agent consists of four components: a cloud simulator, three RL agents, a workload forecaster, and a safety coordinator. Fig. 1 shows the data flow.

IV. Methodology

A. Problem Formulation

We model the cloud resource management problem as a decentralised partially observable Markov Decision Process (Dec-POMDP) [23] with shared observations and independent actions:

- **State** $s_t \in \mathbb{R}^{215}$: Node features (120 dims: 20 nodes $\times$ 6 features—CPU, memory, age, is_booting, is_active, is_draining), job features (80 dims: 20 jobs $\times$ 4 features—service time, wait time, priority, deadline urgency), and global features (15 dims: active VM count, queue length, mean CPU/memory utilisation, forecaster predictions $\hat{d}_{t+h}$ and uncertainties σ_{t+h} for $h \in \{1, 5, 10, 15\}$). All values normalised to $[0, 1]$.
- **Actions:**
  - ScaleOut: $a_t^{\text{so}} \in \{0, 1, 2\}$ (hold, add 1. VM, add 2. VMs)
  - Consolidation: $a_t^{\text{con}} \in \{0, 1\}^{20}$ (per-node drain mask)
  - Scheduling: $a_t^{\text{sch}} \in \{0, \ldots, 4\}$ (priority strategy for the dispatcher)
- **Transition:** $P(s_{t+1}|s_t, a_t)$ is determined by the SimPy discrete-event simulator, which models VM boot delays, job placement, migration overhead, and workload arrival.
- **Rewards:** Agent-specific shaped rewards (Section IV-D).

B. Independent PPO (I-PPO)

Each agent $i \in \{\text{so}, \text{con}, \text{sch}\}$ is trained with Proximal Policy Optimization [1] independently. In I-PPO, agents share the observation s_t but maintain separate policy networks π_θ^i,

value networks V_ϕ^i, and replay buffers. The clipped surrogate objective for agent i is:

$$L_i^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t^i \hat{A}_t^i,\ \text{clip}(r_t^i, 1-\epsilon, 1+\epsilon)\,\hat{A}_t^i\right)\right] \quad (1)$$

where $r_t^i = \frac{\pi_\theta^i(a_t^i|s_t)}{\pi_{\theta_{\text{old}}}^i(a_t^i|s_t)}$ is the importance sampling ratio and $\hat{A}_t^i$ is the Generalized Advantage Estimate (GAE) [6]:

$$\hat{A}_t^i = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}^i, \quad \delta_t^i = r_t^i + \gamma V^i(s_{t+1}) - V^i(s_t) \quad (2)$$

with discount $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, and clip ratio $\epsilon = 0.2$.

The total training loss per agent combines the policy, value, and entropy terms:

$$\mathcal{L}_i = L_i^{\text{CLIP}} - c_1 \|V_\phi^i(s_t) - V_{\text{target}}^i\|^2 + c_2 H[\pi_\theta^i] \quad (3)$$

where $c_1 = 0.5$ (value loss coefficient), c_2 is the entropy coefficient (annealed from 0.01 to 0.001), and $H[\cdot]$ denotes the entropy bonus that encourages exploration.

C. Temporal Hierarchy

A key architectural choice is that agents operate at different frequencies matching the natural timescale of their decisions, as shown in Table I. This is motivated by the observation that capacity planning (ScaleOut) is a slow decision whose effects persist for minutes, while job dispatch (Scheduling) requires immediate, per-step responses.

TABLE I: Temporal hierarchy of agents.

Agent	Freq.	$\lvert A\rvert$	Rationale
Scheduling	Every step (30 s)	5	Jobs arrive continuously
Consolidation	Every 2 steps (1 min)	2^{20}	Trend observation
ScaleOut	Every 10 steps (5 min)	3	VM provisioning

Without temporal decomposition, all three agents would act at every step, leading to compounding action noise and VM thrashing (rapid provision–drain–provision cycles).

D. Reward Functions

Each agent receives a tailored reward signal designed to guide learning towards its specific sub-task while maintaining alignment with global objectives.

ScaleOut:

$$\begin{aligned} r_{\text{so}} = &-\alpha_1 \cdot \text{underProv} - \alpha_2 \cdot \text{overProv} \\ &- \alpha_3 \cdot \text{runCost} - \alpha_4 \cdot \text{bootPenalty} \\ &+ \alpha_5 \cdot \text{proactiveBonus} \end{aligned} \quad (4)$$

Consolidation:

$$\begin{aligned} r_{\text{con}} = &-\beta_1 \cdot \text{runCost} + \beta_2 \cdot \text{slaBonus} \\ &- \beta_3 \cdot \text{migrationPenalty} - \beta_4 \cdot \text{memPressure} \end{aligned} \quad (5)$$

Scheduling:

$$r_{\text{sch}} = -\gamma_1 \cdot \text{meanWaitTime} - \gamma_2 \cdot \text{slaViolation} \quad (6)$$

The reward coefficients are listed in Table II.

TABLE II: Reward function hyperparameters. Values were selected via grid search over $\{0.1, 0.3, 0.5, 1.0, 2.0\}$ on a validation split of Day 1.

Agent	Parameter	Value
ScaleOut	α_1 (under-prov.)	2.0
	α_2 (over-prov.)	1.0
	α_3 (running cost)	0.5
	α_4 (boot penalty)	0.3
	α_5 (proactive bonus)	0.5
Consolidation	β_1 (running cost)	1.0
	β_2 (SLA bonus)	2.0
	β_3 (migration pen.)	0.5
	β_4 (memory press.)	1.0
Scheduling	γ_1 (mean wait)	1.0
	γ_2 (SLA violation)	2.0

E. Workload Forecaster

The forecaster is a Transformer encoder [2] with MC Dropout [3] for uncertainty quantification:

- **Input:** Last 20 time steps, each with 4 features (CPU utilisation, queue length, hour-of-day, day-of-week).
- **Architecture:** 2-layer Transformer encoder, 4 attention heads, 64-dimensional hidden state, sinusoidal positional encoding.
- **Output:** Predictions at 4 horizons $(t{+}1, t{+}5, t{+}10, t{+}15)$ at 3 quantiles (10th, 50th, 90th percentile).
- **Uncertainty:** At inference, dropout is kept active and 30 stochastic forward passes are run. The per-horizon variance across passes provides the uncertainty estimate σ_{t+h}:

$$\sigma_{t+h} = \text{Var}\left[\hat{d}_{t+h}^{(k)}\right]_{k=1}^{30} \quad (7)$$

- **Training loss:** Combined MSE and quantile loss:

$$\mathcal{L}_F = \text{MSE}(\hat{d}, d) + \sum_{q\in\{0.1,0.5,0.9\}} \rho_q(\hat{d}_q, d) \quad (8)$$

where ρ_q is the pinball loss for quantile q.

F. Safety Coordinator

A rule-based post-processing filter enforces seven hard constraints (Table III). This architecture separates *learning* (reward-driven policy improvement) from *safety* (constraint enforcement)—a design principle advocated by Amodei et al. [16] for deploying RL systems safely.

The Safety Coordinator guarantees that the RL agents cannot crash the system, even with a poorly trained or randomly initialised policy. This is critical for safe exploration during training and for deploying in production where stability is non-negotiable.

V. Implementation

A. Cloud Simulator

We built a discrete-event cloud simulator using **SimPy** 4.0 wrapped in a **Gymnasium** 0.29 environment interface. The Gymnasium API enables standardised interaction (reset/step/render) and compatibility with any RL training library. Key parameters:

TABLE III: Safety Coordinator: seven-filter pipeline. Filters are applied sequentially; each filter can modify the action before passing it to the next.

#	Filter	Rule
1	Boot Protection	Never drain a node younger than boot_time + 60 s
2	$N_{\min}$ Floor	Never go below 3 active nodes
3	Uncert. Suppress	If $\sigma_{t+5} > 0.3$, block all drains
3b	High-Load Guard	Suppress drains if mean CPU $>$ 75%, queue $>$ 10, or any node booting
4	Scale-Out Suppress	Block scale-up if $\geq$2 nodes are currently booting
5	Proactive Scale-Out	If $\sigma > 0.3$ AND CPU rising, force +1 VM
6	Queue Emergency	Force +1 VM if queue > 40; force +2 if queue > 120

- **Cluster:** 3–20 VMs with 4 node types (varying CPU/memory capacity).
- **Time step:** 30 seconds. Episode length: 120 steps (1 hour of simulated time).
- **Node lifecycle:** BOOTING (30–90 s) $\rightarrow$ ACTIVE $\rightarrow$ DRAINING (30 s grace period for job migration) $\rightarrow$ TERMINATED.
- **Workload:** Real arrival rates from the Alibaba Cluster Trace 2018 [7], binned at 30-second intervals (2,880 snapshots/-day, 7 days total, 4,023 machines).

A key implementation detail: the `queue_len` metric is captured at its peak *before* the SimPy dispatcher clears the queue each step (`step_peak_queue`), ensuring accurate queue depth reporting for Filter 6 of the Safety Coordinator.

The simulator produces a 215-dimensional observation at each step: 120 node features (20 nodes $\times$ 6 features), 80 job features (20 jobs $\times$ 4 features), and 15 global features (active node count, queue length, mean CPU/memory, forecaster predictions and uncertainties).

B. Agent Architecture

Each agent uses a 3-layer MLP with LayerNorm and orthogonal initialisation [21]:

```
Input(215) -> Linear(512) -> LN -> ReLU
        -> Linear(256) -> LN -> ReLU
        -> Linear(128) -> LN -> ReLU
        -> Output (action logits)
```

The ScaleOut and Scheduling agents use `Categorical` distributions over their discrete action spaces. The Consolidation agent uses independent `Bernoulli` distributions for each of the 20 drain decisions. Orthogonal initialisation with gain $\sqrt{2}$ for hidden layers and gain 0.01 for the output head follows best practices for PPO stability [19].

C. Training Pipeline

Training proceeds in two phases:

Phase 1—Forecaster training (20 min on T4 GPU): The Transformer is trained on Day 1 of the Alibaba trace using combined MSE and quantile loss. Training: 50 epochs with early stopping (patience = 10), Adam optimiser [20] (lr = 10^{-3}, weight decay = 10^{-5}).

Phase 2—RL agents via I-PPO (30 min on T4 GPU): The three agents are trained simultaneously on the simulator with the frozen forecaster:

- Training steps: 100,000+
- PPO rollout buffer: 2,048 steps per update
- PPO epochs per update: 4
- Mini-batch size: 512
- Learning rate: 3×10^{-4} (Adam)
- Entropy coefficient: $0.01 \rightarrow 0.001$ (linear annealing over training)
- EMA reward normalisation [22] (window = 1,000 steps)

All training was performed on Kaggle free-tier (NVIDIA T4 GPU, 16 GB VRAM). Total wall-clock time: $\sim$50 minutes.

D. Software Stack

TABLE IV: Technology stack used in AutoCloud-Agent.

Component	Technology
Simulator	SimPy 4.0 + Gymnasium 0.29
RL agents	PyTorch 2.0 (custom PPO implementation)
Forecaster	PyTorch Transformer + MC Dropout
Language	Python 3.11
Training GPU	Kaggle (NVIDIA T4, 16 GB)
Workload data	Alibaba Cluster Trace 2018 [7]

E. Interactive Demo

The interactive demo (`demo.py`) supports four modes:

- **Standard**: step-by-step walkthrough of AutoCloud-Agent vs. one user-selected baseline.
- **Shootout**: AutoCloud-Agent vs. all five baselines with a ranked results table.
- **Stress**: AutoCloud-Agent vs. a user-selected baseline on an extreme out-of-distribution workload scenario (ramp-up, shock, choppy plateau, or trough + recovery).
- **Ablation**: 3-agent I-PPO vs. Single-Agent PPO, illustrating the value of decomposition.

VI. Evaluation

A. Metrics

We evaluate all methods using four complementary metrics:

TABLE V: Evaluation metrics.

Metric	Definition
SLA Rate	Fraction of steps where P95 latency $<$ 500 ms
Cost Efficiency	$1 - \frac{\text{actual cost}}{\text{max possible cost}}$
CPU Utilisation	Mean CPU usage across active nodes
Node Stability	$1 - \frac{\text{std}(N_{\text{active}})}{\text{mean}(N_{\text{active}})}$

B. Baselines

We compare against six baselines spanning industry practice, classical control theory, and RL (Table VI).

TABLE VI: Baseline methods. All baselines are implemented faithfully from their original descriptions and evaluated on the same simulator.

Category	Method	Description
Industry	KubernetesHPA [8]	k8s HPA with 10% dead-band
Industry	AWSTargetTracking [9]	Asymmetric cooldowns
Control	MPCController [4]	5-step MPC with EWMA
Rule	ThresholdReactive	CPU >80%: add; <30%: drain
RL ablation	SingleAgentPPO	One PPO, joint action space
Lower bound	StaticN	Fixed 10 nodes

C. Experimental Setup

All methods are evaluated on the Alibaba Cluster Trace 2018 using 5 episodes $\times$ 3 random seeds = 15 runs per method. Results are averaged and reported with standard errors. The RL agents use frozen checkpoints from Phase 2 training.

D. Results

Results are shown in Table VII.

TABLE VII: Evaluation results on Alibaba Cluster Trace 2018 (mean over 15 runs). Best result per column is **bolded**.

Method	SLA	Cost Eff.	CPU Util.	Stability
AutoCloud	**100%**	**.962**	**.552**	.889
MPC	100%	.962	.552	**.941**
Threshold	100%	.955	.483	.822
StaticN	100%	.938	.333	1.00
K8s HPA	100%	.930	.312	.842
AWS TT	100%	.928	.295	.876
SinglePPO	100%	.924	.413	.794

E. Analysis

Cost efficiency: AutoCloud-Agent matches MPC (0.962), the best classical method. It outperforms Kubernetes HPA by 3.4% and AWS Target Tracking by 3.7%. Unlike MPC, AutoCloud-Agent requires no forecasting model specification—the agent learns an implicit model through interaction.

CPU utilisation: AutoCloud-Agent achieves 55.2%, nearly double that of AWS Target Tracking (29.5%). Higher utilisation means fewer idle resources, directly translating to lower operational cost. This improvement stems from the Consolidation agent's ability to identify and drain underutilised VMs that threshold methods would keep running.

Multi-agent vs. single-agent: SingleAgentPPO (0.924 cost efficiency, 41.3% CPU) significantly underperforms AutoCloud-Agent (0.962, 55.2%) across all metrics. This confirms the value of multi-agent decomposition and temporal hierarchy (see ablation below).

Stability trade-off: MPC achieves higher stability (0.941 vs. 0.889) because its control-theoretic approach produces naturally smooth node-count trajectories. The RL agent trades some stability for higher responsiveness to demand changes, which is appropriate given the Safety Coordinator prevents dangerous oscillations.

SLA compliance: All methods achieve 100% SLA on this trace. The differentiator is cost—how cheaply each method maintains SLA compliance.

F. Ablation: Why Multi-Agent Decomposition?

The SingleAgentPPO baseline serves as an ablation study. A single agent must control a joint action space of $3 \times 2^{20} \times 5 \approx 15$ million combinations simultaneously. The I-PPO decomposition yields:

1) **+4.1% cost efficiency** (0.962 vs. 0.924): Smaller per-agent action spaces enable faster policy convergence.
2) **+33.7% CPU utilisation** (55.2% vs. 41.3%): Agent-specific reward shaping provides clearer learning signals.
3) **Temporal decomposition**: Matching decision frequency to task timescale prevents the agent from confusing fast (scheduling) and slow (scale-out) decisions.
4) **Credit assignment**: When an SLA violation occurs, we can identify which agent's decision was responsible, simplifying debugging.

G. Stress Testing

We evaluate robustness under four out-of-distribution (OOD) workload patterns not seen during training:

1) **Ramp-Up:** Linear increase $1\times \rightarrow 3\times$ over 120 steps. Tests continuous provisioning under sustained growth.
2) **Early Shock:** Steady $0.8\times$ then sudden $4\times$ spike at step 15. Tests reactive/proactive scaling.
3) **Choppy Plateau:** Sustained $2.5\times$ with ± 0.3 random noise. Tests stability under persistent high-load fluctuations.
4) **Trough & Recovery:** Drop to $0.3\times$, then ramp to $2.5\times$. Tests conservative consolidation and rapid recovery.

The Safety Coordinator's proactive scale-out filter (Filter 5) and queue emergency filter (Filter 6) are critical for the Early Shock scenario, forcing immediate node addition when the forecaster's uncertainty spikes and the queue exceeds 40 pending jobs. In all four scenarios, AutoCloud-Agent maintained 100% SLA compliance while the reactive baselines incurred transient SLA violations during spike transitions.

VII. Key Design Decisions

1) **Gymnasium API:** Wrapping SimPy in Gymnasium enables standardised RL training, fair baseline comparison, and compatibility with existing RL tooling.
2) **Safety as a separate module:** Hard constraints are enforced in a post-processing filter rather than embedded in the reward function. This *guarantees* safety regardless of policy quality—a distinction from reward-based penalty approaches where violations are merely *discouraged* [16].
3) **MC Dropout over ensembles:** 30 forward passes through one model vs. training 5+ ensemble members. Comparable uncertainty calibration at $5\times$ lower compute [3].

4) **EMA reward normalisation:** Running mean/variance normalisation stabilises training across episodes with varying reward magnitudes [22].
5) **Real trace data:** The Alibaba 2018 trace (4,023 machines, 7 days) ensures evaluation reflects actual cloud workload patterns, not synthetic distributions.

VIII. Limitations and Future Work

Limitations: (i) Homogeneous VMs—real clouds offer heterogeneous instance types with different CPU/memory/price ratios; (ii) single-cluster scope—production systems manage thousands of clusters across regions; (iii) simplified networking—no cross-rack latency or bandwidth modelling; (iv) static reward weights—adaptive reward shaping could improve performance on non-stationary workloads; (v) single-day training—multi-day training with domain randomisation would improve generalisation.

Future Work: Heterogeneous VM types with bin-packing strategies; multi-cluster federation with hierarchical agents; vertical scaling (resizing individual VMs); transfer learning across different cloud traces (Azure, Google Borg); spot/preemptible instance awareness (price-aware scaling); and integration with container orchestrators like Kubernetes for real-world deployment.

IX. Conclusion

We presented AutoCloud-Agent, a multi-agent reinforcement learning system for cloud resource management that simultaneously optimises for elasticity and cost minimisation. By decomposing the problem into three independent sub-agents with temporal hierarchy, incorporating uncertainty-aware forecasting via Transformer + MC Dropout, and enforcing hard safety constraints through a seven-filter coordinator, AutoCloud-Agent achieves state-of-the-art cost efficiency (0.962) and CPU utilisation (55.2%) while maintaining 100% SLA compliance on the Alibaba Cluster Trace 2018. The multi-agent I-PPO approach outperforms single-agent RL by 4.1% on cost and 33.7% on CPU utilisation, validating the decomposition strategy. The system matches the best classical method (MPC) while offering greater adaptability to non-stationary workloads and requiring no hand-tuned forecasting model.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[3] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2016, pp. 1050–1059.

[4] Q. Zhang, Y. Liu, J. Tan, W. Zheng, and Y. Zhang, "AWARE: Automate Workload Autoscaling with Reinforcement Learning in Production Cloud Systems," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2023, pp. 387–402.

[5] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource Management with Deep Reinforcement Learning," in *Proc. ACM Workshop Hot Topics in Networks (HotNets)*, 2016, pp. 50–56.

[6] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[7] Alibaba Group, "Cluster Trace Program," https://github.com/alibaba/clusterdata, 2018.

[8] Kubernetes, "Horizontal Pod Autoscaler," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, accessed Apr. 2026.

[9] Amazon Web Services, "Target Tracking Scaling Policies for Amazon EC2 Auto Scaling," https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html, accessed Apr. 2026.

[10] Google Cloud, "Autoscaling Groups of Instances," https://cloud.google.com/compute/docs/autoscaler, accessed Apr. 2026.

[11] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An Intelligent Fine-Grained Resource Management Framework for SLO-Oriented Microservices," in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 805–825.

[12] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes, "Borg: The Next Generation," in *Proc. European Conf. Computer Systems (EuroSys)*, 2020.

[13] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 6379–6390.

[14] C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. S. Torr, M. Sun, and S. Whiteson, "Is Independent Learning All You Need in the StarCraft Multi-Agent Challenge?" *arXiv preprint arXiv:2011.09533*, 2020.

[15] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 6402–6413.

[16] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete Problems in AI Safety," *arXiv preprint arXiv:1606.06565*, 2016.

[17] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments," *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.

[18] C. E. Garcia, D. M. Prett, and B. L. Morari, "Model Predictive Control: Theory and Practice—a Survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.

[19] M. Andrychowicz et al., "What Matters in On-Policy Reinforcement Learning? A Large-Scale Empirical Study," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[20] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[21] H. Hu, Z. Ye, and Y. He, "Understanding the Role of Orthogonal Initialization in Deep Networks," *arXiv preprint arXiv:2004.05867*, 2020.

[22] B. P. Welford, "Note on a Method for Calculating Corrected Sums of Squares and Products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[23] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*, Springer, 2016.